Design Document

Eduvantage AI · Student Success Intelligence · Version 1.0 · 6 October 2026

1. Purpose and scope

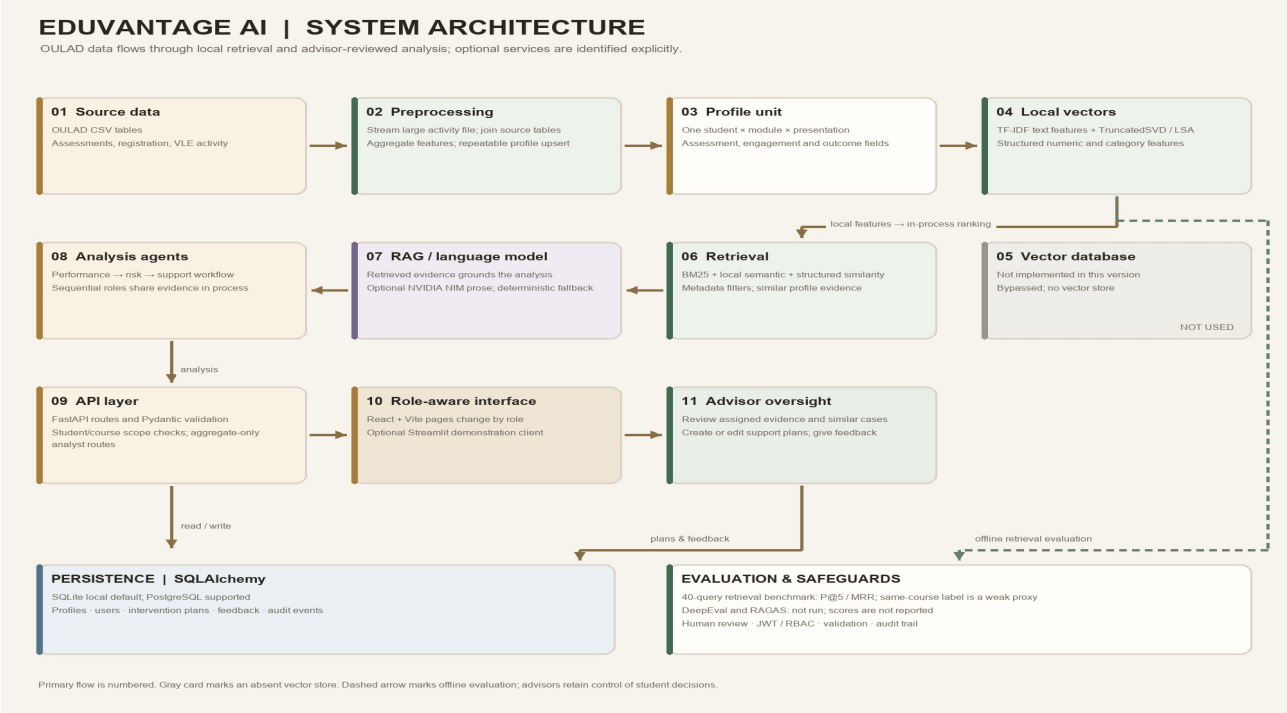
An advisor-facing student-success prototype that organizes historical academic and learning-activity evidence, similarity search, screening indicators, and human-owned support plans. The system is decision support, not an automated enrollment, grading, or intervention decision maker.

2. Users and workflows

- Advisors and faculty review student-course profiles, apply cohort filters, inspect evidence, ask the assistant, and maintain intervention plans.
- Administrators manage data and workspace operations; analysts inspect course trends, model estimates, exports, and evaluation notes.
- Main workflow: authenticate → dashboard → filter/prioritize → inspect profile and similar cases → review recommendation → record plan → submit feedback.

3. Architecture and components

React/Vite provides the browser UI. FastAPI provides REST endpoints, Pydantic validation, authentication and role enforcement. SQLAlchemy persists user, student, intervention, feedback, and audit tables to SQLite locally or PostgreSQL by DATABASE_URL. Local retrieval and deterministic scoring run in the backend; optional NVIDIA NIM prose and judging run server-side.



4. Data model and processing

The OULAD importer creates one student × module × presentation record. It joins studentInfo, studentRegistration, studentAssessment, assessments, studentVle, vle, and courses. It streams the large activity CSV, aggregates clicks and active weeks, derives assessment means/trends and completion, then upserts records idempotently. See dataset-details.pdf and dataset-details.md for counts and assumptions.

5. Retrieval, agents, and scoring

Hybrid case retrieval blends BM25 keyword ranking with local TF-IDF/TruncatedSVD semantic text and structured feature similarity. Metadata filters can constrain module/presentation, region, education, age band, credits, and previous attempts. A shortlist reduces latent fitting cost. Three deterministic agents report performance, risk factors, and suggested advisor actions. Optional NIM produces evidence-constrained prose and a separate judge; numeric risk fields stay deterministic.

6. Prediction and feedback

A RandomForestRegressor estimates assessment score and a RandomForestClassifier estimates withdrawal label from available profile features. Holdout metrics are calculated on local records. Advisors can submit feedback; the current system stores ratings for review but does not automatically retrain from feedback.

7. Security, privacy, and oversight

- Password hashing, signed expiring JWT bearer tokens, role checks, request validation, and audit events.
- The NVIDIA key is configured in ignored backend/.env and is never sent to the frontend.
- OULAD IDs are anonymized; row-level education data still requires controlled access and institutional governance.
- Risk scores are heuristic indicators, not calibrated probabilities or causal explanations. Human review is required.

8. Limitations and next work

The benchmark uses same-course membership as a weak relevance label; keyword-only currently exceeds the hybrid score. Models lack external validation, calibration, subgroup fairness review, and prospective evidence. DeepEval output is pending. Next steps are advisor-labeled retrieval relevance data, repeatable API regression tests, institutional privacy/security review, and prospective model evaluation before any real operational use.

9. Technology selection and justification

Technology	Selection and rationale
Language / API	Python 3.11+, FastAPI, Pydantic, Uvicorn for typed REST validation and data/ML ecosystem.
UI / database	React 18 + Vite primary site; optional Streamlit demo client. SQLAlchemy with SQLite locally and PostgreSQL as an option.
LLM	Optional NVIDIA NIM using configured NVIDIA_MODEL. It drafts evidence-grounded prose and judge feedback; deterministic core remains available.
Semantic representation	Local TF-IDF + TruncatedSVD/LSA, not a pretrained embedding model. No vector database is used; candidates are ranked in process.
ML	scikit-learn random-forest demonstrations and local retrieval evaluation.

10. Retrieval strategy and RAG boundaries

The retrieval document is one structured student × module × presentation profile. The importer streams and aggregates the source tables; it does not split prose into overlapping token chunks. Retrieval applies course/presentation, region, education, age-band, credits, and attempts filters, then combines BM25 keyword relevance, local TF-IDF/LSA text similarity, and structured feature similarity. Selected cases ground the analysis workflow. No pretrained embedding endpoint or vector store exists in this version.

11. Agent roles and communication

The performance analyst summarizes assessment and activity evidence; the risk assessor assigns a heuristic band and factors; the intervention recommender proposes human-reviewed actions using prior outputs and retrieved cases. FastAPI calls these roles sequentially in one request, passing typed dictionaries/results in process. Optional NIM prose/judging is supplemental and cannot change deterministic numeric risk fields.

12. Evaluation strategy and current metrics

The 40-query retrieval benchmark reports Precision@5 and MRR against same-course membership as a weak relevance proxy: hybrid 0.450 / 0.681; semantic-only 0.395 / 0.617; keyword-only 0.475 / 0.729. The proxy currently favors keyword-only. Optional DeepEval covers faithfulness, answer relevance, context relevance, and custom support quality, but no aggregate DeepEval run is included. Manual sample cases are in docs/sample-test-cases.md. Do not claim pending evaluation as a result.

13. Local deployment requirement

Run the backend with FastAPI/Uvicorn from backend/ (`uvicorn app.main:app --reload`). Run the full React/Vite interface from frontend/ (`npm run dev`) or the optional Streamlit demo from the repository root (`streamlit run frontend.py`). Both call the same API. Python dependencies are listed in backend/requirements.txt. Environment and API-key setup is in README.md; secrets stay out of source control.

14. Challenges, trade-offs, and limitations

Streaming makes the 10.6M-row VLE import practical on a local machine. Local TF-IDF/LSA avoids an external embedding service, but it is less semantically general than a pretrained encoder. No vector DB avoids infrastructure, but in-process ranking is less scalable. Historical model estimates are not operationally validated; feedback is stored but does not automatically retrain models. Risk indicators require human review.

15. Persistent role scopes

A user_access table stores advisor student grants and faculty course grants. Registration does not accept user-selected roles; an administrator assigns roles and scopes. Server-side queries constrain individual record operations and student CSV exports. Analysts receive aggregate reports only. Administrators can update roles and grant or revoke scopes from the UI; changes persist across application restarts.